



UNIVERSIDADE DO VALE DO ITAJAÍ

Escola Politécnica

RELATÓRIO TÉCNICO

Projeto de Circuitos Combinacionais usando VHDL

Autor(es): Gabriel Antônio Sereia

Disciplina: Projeto de Sistemas Digitais

Professor: Douglas Rossi de Melo

Itajaí – SC

28 de setembro de 2026

Sumário

1	Introdução	2
2	Objetivos	2
3	Procedimento Experimental	2
4	Circuito 1 – Multiplexador 2x1 de 1 bit	3
4.1	Descrição do Circuito	3
4.2	Código VHDL	4
4.3	Testbench	4
4.4	Resultados da Simulação	4
4.5	Discussão dos Resultados	5
5	Circuito 2 – Demultiplexador 1x2 de 1 bit	5
5.1	Descrição do Circuito	5
5.2	Código VHDL	6
5.3	Testbench	7
5.4	Resultados da Simulação	8
5.5	Discussão dos Resultados	9
6	Circuito 3 – Conversor Binário para One-Hot	9
6.1	Descrição do Circuito	9
6.2	Código VHDL	9
6.3	Testbench	10
6.4	Resultados da Simulação	12
6.5	Discussão dos Resultados	13
7	Circuito 4 – Conversor One-Hot para Binário	13
7.1	Descrição do Circuito	13
7.2	Código VHDL	14
7.3	Testbench	15
7.4	Resultados da Simulação	17
7.5	Discussão dos Resultados	17
8	Conclusão	17

1 INTRODUÇÃO

Este relatório apresenta o desenvolvimento e a verificação de circuitos combinacionais descritos em VHDL, como parte da Avaliação 2 da disciplina de Projeto de Sistemas Digitais. O trabalho tem como objetivo aplicar os conceitos de lógica digital e projeto de circuitos combinacionais por meio da descrição de hardware e da realização de simulações em ambiente EDA Playground.

A atividade contempla quatro circuitos digitais: um multiplexador 2x1 com canal de dados de 1 bit, utilizado como circuito de referência; um demultiplexador 1x2 com canal de dados de 1 bit; um circuito de conversão entre representação binária de 2 bits e representação one-hot de 4 bits; e um circuito de conversão entre representação one-hot de 4 bits e representação binária de 2 bits.

Para cada circuito implementado, foi desenvolvido um testbench responsável por aplicar diferentes combinações de entrada e verificar se as saídas obtidas correspondem ao comportamento esperado. As simulações permitem validar o funcionamento dos circuitos antes de sua implementação física, além de possibilitar a análise das formas de onda e dos resultados apresentados no console.

Dessa forma, o trabalho busca consolidar os conhecimentos relacionados à descrição de circuitos combinacionais em VHDL, à elaboração de testbenches e à interpretação dos resultados obtidos por meio de simulação.

2 OBJETIVOS

O objetivo desta avaliação é aplicar os conceitos de projeto de circuitos combinacionais por meio da descrição e simulação de circuitos digitais utilizando VHDL. Para isso, busca-se:

- implementar um demultiplexador 1x2 com canal de dados de 1 bit;
- implementar um circuito de conversão de 2 bits binários para uma representação one-hot de 4 bits;
- implementar um circuito de conversão de 4 bits one-hot para 2 bits binários;
- desenvolver testbenches para verificar o funcionamento dos circuitos implementados;
- analisar as formas de onda e os resultados obtidos nas simulações realizadas no EDA Playground.

3 PROCEDIMENTO EXPERIMENTAL

Para o desenvolvimento e verificação dos circuitos combinacionais, foram seguidas as seguintes etapas:

1. Analisou-se o enunciado e definiu-se a tabela verdade de cada circuito.
2. A partir das tabelas verdade, foram determinadas as expressões lógicas correspondentes às saídas.
3. Os circuitos combinacionais foram descritos em VHDL, seguindo o guia de estilo de codificação definido para a atividade.
4. Para cada circuito implementado, foi desenvolvido um testbench para aplicar as combinações de entrada e verificar as respectivas saídas.
5. Os códigos dos circuitos e dos testbenches foram executados e simulados no ambiente EDA Playground.
6. Foram analisadas as formas de onda geradas durante as simulações para verificar o comportamento dos circuitos.
7. Os resultados das simulações foram verificados por meio das mensagens apresentadas no console, utilizando asserções nos testbenches.

4 CIRCUITO 1 – MULTIPLEXADOR 2X1 DE 1 BIT

4.1 Descrição do Circuito

O primeiro circuito consiste em um multiplexador 2x1 com canal de dados de 1 bit. O circuito possui duas entradas de dados, A e B , uma entrada de seleção SEL e uma saída S . O sinal de seleção determina qual das duas entradas será encaminhada para a saída.

Quando $SEL = 0$, a saída S recebe o valor da entrada A . Quando $SEL = 1$, a saída S recebe o valor da entrada B .

A tabela verdade do multiplexador é apresentada na Tabela 1.

Tabela 1: Tabela verdade do multiplexador 2x1.

SEL	A	B	S
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

4.2 Código VHDL

O multiplexador foi descrito em VHDL utilizando uma arquitetura comportamental baseada em um processo sensível às entradas do circuito. A seleção da entrada encaminhada para a saída é realizada por meio de uma estrutura condicional `if`.

```
library ieee;
use ieee.std_logic_1164.all;

entity mux2_1bit is
port ( i_SEL : in  std_logic;
      i_A   : in  std_logic;
      i_B   : in  std_logic;
      o_S   : out std_logic);
end mux2_1bit;

architecture arch_1 of mux2_1bit is
begin
  process(i_SEL, i_A, i_B)
  begin
    if (i_SEL='0') then
      o_S <= i_A;
    else
      o_S <= i_B;
    end if;
  end process;
end arch_1;
```

4.3 Testbench

O testbench fornecido para o circuito foi utilizado para verificar o comportamento do multiplexador. Foram aplicadas diferentes combinações das entradas *A*, *B* e *SEL*, verificando se a saída *S* corresponde à entrada selecionada.

4.4 Resultados da Simulação

A simulação foi realizada no ambiente EDA Playground. A Figura 1 apresenta as formas de onda obtidas durante a execução do testbench.

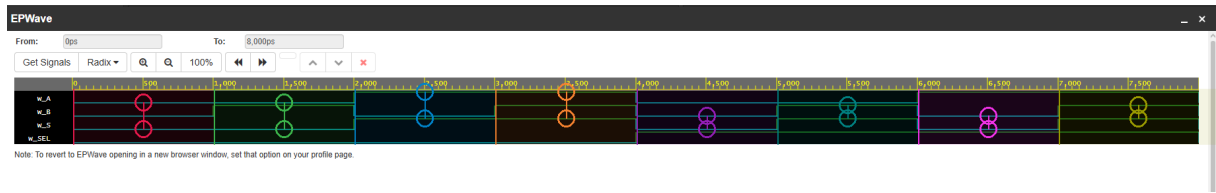


Figura 1: Formas de onda da simulação do multiplexador 2x1.

Na seção em vermelho, podemos notar que: $SEL=0$, $A=0$, $B=0$, temos $S=0$.

Na seção em verde, podemos notar que: $SEL=0$, $A=0$, $B=1$, temos $S=0$.

Na seção em azul, podemos notar que: $SEL=0$, $A=1$, $B=0$, temos $S=1$.

Na seção em laranja, podemos notar que: $SEL=0$, $A=1$, $B=1$, temos $S=1$.

Na seção em roxo, podemos notar que: $SEL=1$, $A=0$, $B=0$, temos $S=0$.

Na seção em azul-petróleo (teal), podemos notar que: $SEL=1$, $A=0$, $B=1$, temos $S=1$.

Na seção em magenta, podemos notar que: $SEL=1$, $A=1$, $B=0$, temos $S=0$.

Na seção em amarelo-oliva, podemos notar que: $SEL=1$, $A=1$, $B=1$, temos $S=1$.

4.5 Discussão dos Resultados

A simulação apresentou o comportamento esperado para o multiplexador 2x1. Para $SEL = 0$, a saída S acompanhou o valor da entrada A , enquanto para $SEL = 1$, a saída S acompanhou o valor da entrada B . Os resultados observados nas formas de onda e no console confirmam o funcionamento correto do circuito.

5 CIRCUITO 2 – DEMULTIPLEXADOR 1X2 DE 1 BIT

5.1 Descrição do Circuito

O segundo circuito consiste em um demultiplexador 1x2 com canal de dados de 1 bit. O circuito possui uma entrada de dados A , uma entrada de seleção SEL e duas saídas, $S0$ e $S1$. O sinal de seleção determina para qual das duas saídas o sinal de entrada será encaminhado.

Quando $SEL = 0$, o valor da entrada A é encaminhado para a saída $S0$, enquanto $S1$ permanece em nível lógico baixo. Quando $SEL = 1$, o valor de A é encaminhado para $S1$, enquanto $S0$ permanece em nível lógico baixo.

As expressões lógicas das saídas são:

$$S0 = A \cdot \overline{SEL}$$

$$S1 = A \cdot SEL$$

A tabela verdade do demultiplexador é apresentada na Tabela 2.

Tabela 2: Tabela verdade do demultiplexador 1x2.

SEL	A	S0	S1
0	0	0	0
0	1	1	0
1	0	0	0
1	1	0	1

5.2 Código VHDL

O demultiplexador foi descrito em VHDL utilizando expressões lógicas para determinar o valor de cada saída. A saída $S0$ é ativada quando a entrada A está em nível lógico alto e o sinal de seleção está em nível lógico baixo. De forma complementar, a saída $S1$ é ativada quando A está em nível lógico alto e SEL está em nível lógico alto.

```
library ieee;
use ieee.std_logic_1164.all;

-- S0 = A and not SEL
-- S1 = A and SEL

entity demux1_2bit is
port ( i_SEL : in  std_logic;
      i_A    : in  std_logic;
      o_S0   : out std_logic;
      o_S1   : out std_logic);
end demux1_2bit;

architecture arch_1 of demux1_2bit is
begin
    o_S0 <= (i_A and not i_SEL);
    o_S1 <= (i_A and i_SEL);
end arch_1;

configuration config_demux1_2bit of demux1_2bit is
    for arch_1
    end for;
end config_demux1_2bit;
```

```
end;
```

5.3 Testbench

O testbench foi desenvolvido para verificar todas as combinações possíveis entre a entrada de dados *A* e o sinal de seleção *SEL*. Para cada combinação, as saídas *S0* e *S1* são comparadas com os valores esperados por meio de comandos `assert`.

```
library ieee;
use ieee.std_logic_1164.all;
```

```
entity testbench is
-- empty
end testbench;
```

```
architecture tb of testbench is
```

```
-- DUT component
component demux1_2bit is
port ( i_SEL : in  std_logic;  -- selector
       i_A   : in  std_logic;  -- data input
       o_S0  : out std_logic;  -- data output
       o_S1  : out std_logic); -- data output
end component;
```

```
signal w_SEL, w_A, w_S0, w_S1 : std_logic;
```

```
begin
```

```
    u_DUT : demux1_2bit port map (i_SEL => w_SEL,
                                   i_A  => w_A,
                                   o_S0 => w_S0,
                                   o_S1 => w_S1);
```

```
process
```

```
begin
```

```
    w_SEL <= '0';
```

```
    w_A   <= '0';
```

```
    wait for 1 ns;
```

```
    assert (w_S0='0' and w_S1='0') report "Fail @ 000" severity error;
```

```
    w_SEL <= '0';
```

```

w_A    <= '1';
wait for 1 ns;
    assert(w_S0='1' and w_S1='0') report "Fail @ 000" severity error;

    w_SEL <= '1';
w_A    <= '0';
wait for 1 ns;
    assert(w_S0='0' and w_S1='0') report "Fail @ 000" severity error;

w_SEL <= '1';
w_A    <= '1';
wait for 1 ns;
    assert(w_S0='0' and w_S1='1') report "Fail @ 000" severity error;

-- Clear inputs
w_SEL    <= '0';
w_A      <= '0';
assert false report "Test done." severity note;
    wait;
end process;

end tb;

```

5.4 Resultados da Simulação

A simulação foi realizada no ambiente EDA Playground. A Figura 2 apresenta as formas de onda obtidas durante a execução do testbench.

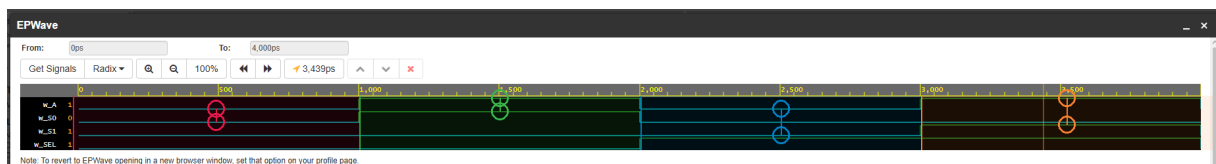


Figura 2: Formas de onda da simulação do demultiplexador 1x2.

Na seção em vermelho, podemos notar que: SEL=0, A=0, temos S0=0 e S1=0.

Na seção em verde, podemos notar que: SEL=0, A=1, temos S0=1 e S1=0.

Na seção em azul, podemos notar que: SEL=1, A=0, temos S0=0 e S1=0.

Na seção em laranja, podemos notar que: SEL=1, A=1, temos S0=0 e S1=1.

5.5 Discussão dos Resultados

A simulação apresentou o comportamento esperado para o demultiplexador 1x2. Para $SEL = 0$, o valor da entrada A foi corretamente encaminhado para a saída $S0$, permanecendo $S1$ em nível lógico baixo. Para $SEL = 1$, o valor de A foi encaminhado para $S1$, permanecendo $S0$ em nível lógico baixo. Os resultados observados nas formas de onda e no console confirmam o funcionamento correto do circuito.

6 CIRCUITO 3 – CONVERSOR BINÁRIO PARA ONE-HOT

6.1 Descrição do Circuito

O terceiro circuito, conforme a nomenclatura adotada no enunciado da avaliação, realiza a conversão entre uma representação one-hot de 4 bits e uma representação binária de 2 bits. O circuito possui quatro entradas, $Y0$, $Y1$, $Y2$ e $Y3$, das quais apenas uma deve estar em nível lógico alto a cada instante (codificação one-hot), e duas saídas, $S0$ e $S1$, que representam o índice da entrada ativa em formato binário, sendo $S1$ o bit mais significativo (MSB) e $S0$ o bit menos significativo (LSB).

As expressões lógicas das saídas são:

$$S0 = Y1 + Y3$$

$$S1 = Y2 + Y3$$

A tabela verdade do circuito é apresentada na Tabela 3.

Tabela 3: Tabela verdade do conversor one-hot para binário.

Y0	Y1	Y2	Y3	S1	S0
1	0	0	0	0	0
0	1	0	0	0	1
0	0	1	0	1	0
0	0	0	1	1	1

6.2 Código VHDL

O circuito foi descrito em VHDL utilizando expressões lógicas simples, obtidas diretamente das expressões booleanas de $S0$ e $S1$.

```
library IEEE;  
use IEEE.std_logic_1164.all;
```

```

-- recebe 4 entradas one-hot e transforma em 2 bits binarios

-- S0 = Y1 + Y3
-- S1 = Y2 + Y3

entity encoder4_2bit is
port ( i_Y0 : in std_logic;
       i_Y1 : in std_logic;
       i_Y2 : in std_logic;
       i_Y3 : in std_logic;
       o_S0 : out std_logic;
       o_S1 : out std_logic);
end encoder4_2bit;

architecture arch_1 of encoder4_2bit is
begin
    o_S0 <= i_Y1 or i_Y3;
    o_S1 <= i_Y2 or i_Y3;
end arch_1;

configuration config_encoder4_2bit of encoder4_2bit is
    for arch_1
    end for;
end;

```

6.3 Testbench

O testbench foi desenvolvido para verificar as quatro combinações one-hot possíveis, ativando uma entrada por vez e comparando as saídas $S0$ e $S1$ com os valores esperados por meio de comandos `assert`.

```

library IEEE;
use IEEE.std_logic_1164.all;

entity testbench is
-- empty
end testbench;

architecture tb of testbench is

```

```

-- DUT encoder4_2bit
component encoder4_2bit is
port ( i_Y0 : in std_logic;    -- data input
      i_Y1 : in std_logic;    -- data input
      i_Y2 : in std_logic;    -- data input
      i_Y3 : in std_logic;    -- data input
      o_S0 : out std_logic;    -- data output
      o_S1 : out std_logic); -- data output
end component;

signal w_Y0, w_Y1, w_Y2, w_Y3, w_S0, w_S1 : std_logic;

begin
    u_DUT : encoder4_2bit port map (i_Y0 => w_Y0,
                                    i_Y1 => w_Y1,
                                    i_Y2 => w_Y2,
                                    i_Y3 => w_Y3,
                                    o_S0 => w_S0,
                                    o_S1 => w_S1);

    process
    begin

        -- Teste Y0 = 1 -> S1S0 = 00
        w_Y0 <= '1';
        w_Y1 <= '0';
        w_Y2 <= '0';
        w_Y3 <= '0';
        wait for 1 ns;
        assert(w_S0='0' and w_S1='0') report "Fail @ Y0" severity error;

        -- Teste Y1 = 1 -> S1S0 = 01
        w_Y0 <= '0';
        w_Y1 <= '1';
        w_Y2 <= '0';
        w_Y3 <= '0';
        wait for 1 ns;
        assert(w_S0='1' and w_S1='0') report "Fail @ Y1" severity error;
    end process;
end;

```

```

-- Teste Y2 = 1 -> S1S0 = 10
w_Y0 <= '0';
w_Y1 <= '0';
w_Y2 <= '1';
w_Y3 <= '0';
wait for 1 ns;
assert(w_S0='0' and w_S1='1') report "Fail @ Y2" severity error;

-- Teste Y3 = 1 -> S1S0 = 11
w_Y0 <= '0';
w_Y1 <= '0';
w_Y2 <= '0';
w_Y3 <= '1';
wait for 1 ns;
assert(w_S0='1' and w_S1='1') report "Fail @ Y3" severity error;

-- Clear inputs
w_Y0 <= '0';
w_Y1 <= '0';
w_Y2 <= '0';
w_Y3 <= '0';
assert false report "Test done." severity note;
wait;
end process;

end tb;

```

6.4 Resultados da Simulação

A simulação foi realizada no ambiente EDA Playground. A Figura 3 apresenta as formas de onda obtidas durante a execução do testbench.

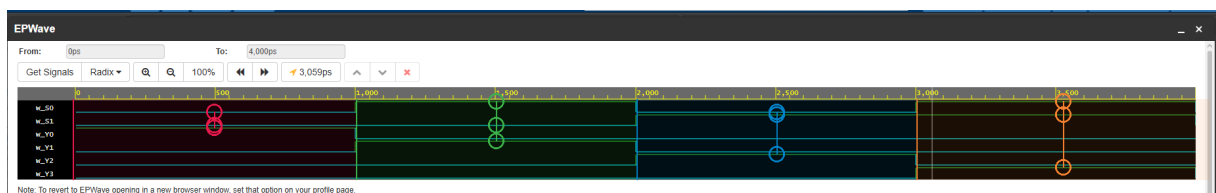


Figura 3: Formas de onda da simulação do conversor one-hot para binário.

Na seção em vermelho, podemos notar que: $Y0=1$, temos $S1S0=00$.

Na seção em verde, podemos notar que: $Y1=1$, temos $S1S0=01$.

Na seção em azul, podemos notar que: $Y2=1$, temos $S1S0=10$.

Na seção em laranja, podemos notar que: $Y3=1$, temos $S1S0=11$.

O resultado apresentado no console também foi analisado para verificar a execução dos testes.

6.5 Discussão dos Resultados

A simulação apresentou o comportamento esperado para o circuito. Cada uma das quatro entradas one-hot ($Y0$ a $Y3$), quando ativada individualmente, produziu a combinação binária correspondente nas saídas $S1S0$. Os resultados observados nas formas de onda e no console confirmam o funcionamento correto do circuito.

7 CIRCUITO 4 – CONVERSOR ONE-HOT PARA BINÁRIO

7.1 Descrição do Circuito

O quarto circuito, conforme a nomenclatura adotada no enunciado da avaliação, realiza a conversão entre uma representação binária de 2 bits e uma representação one-hot de 4 bits. O circuito possui duas entradas, $S0$ e $S1$, sendo $S1$ o bit mais significativo (MSB) e $S0$ o bit menos significativo (LSB), e quatro saídas, $Y0$, $Y1$, $Y2$ e $Y3$, das quais apenas uma é ativada em nível lógico alto de acordo com o valor binário formado por $S1S0$.

As expressões lógicas das saídas são:

$$Y0 = \overline{S1} \cdot \overline{S0}$$

$$Y1 = \overline{S1} \cdot S0$$

$$Y2 = S1 \cdot \overline{S0}$$

$$Y3 = S1 \cdot S0$$

A tabela verdade do circuito é apresentada na Tabela 4.

Tabela 4: Tabela verdade do conversor binário para one-hot.

S1	S0	Y0	Y1	Y2	Y3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

7.2 Código VHDL

O circuito foi descrito em VHDL utilizando expressões lógicas simples, obtidas diretamente das expressões booleanas de $Y0$, $Y1$, $Y2$ e $Y3$.

```
library IEEE;
use IEEE.std_logic_1164.all;

-- recebe 2 bits binarios e transforma em 4 saidas one-hot

-- Y0 = not S1 and not S0
-- Y1 = not S1 and S0
-- Y2 = S1 and not S0
-- Y3 = S1 and S0

entity decoder_2_4bits is
port ( i_S0 : in  std_logic;
      i_S1 : in  std_logic;
      o_Y0 : out std_logic;
      o_Y1 : out std_logic;
      o_Y2 : out std_logic;
      o_Y3 : out std_logic);
end decoder_2_4bits;

architecture arch_1 of decoder_2_4bits is
begin
    o_Y0 <= not i_S1 and not i_S0;
    o_Y1 <= not i_S1 and i_S0;
    o_Y2 <= i_S1 and not i_S0;
    o_Y3 <= i_S1 and i_S0;
end arch_1;

configuration config_decoder_2_4bits of decoder_2_4bits is
```

```

        for arch_1
        end for;
end;

```

7.3 Testbench

O testbench foi desenvolvido para verificar as quatro combinações possíveis de $S1S0$, comparando as quatro saídas com os valores esperados por meio de comandos `assert`.

```

library IEEE;
use IEEE.std_logic_1164.all;

entity testbench is
-- empty
end testbench;

architecture tb of testbench is

-- DUT decoder_2_4bits
component decoder_2_4bits is
port ( i_S0 : in  std_logic; -- data input
       i_S1 : in  std_logic; -- data input
       o_Y0 : out std_logic; -- data output
       o_Y1 : out std_logic; -- data output
       o_Y2 : out std_logic; -- data output
       o_Y3 : out std_logic); -- data output
end component;

signal w_S0, w_S1, w_Y0, w_Y1, w_Y2, w_Y3 : std_logic;

begin

    u_DUT : decoder_2_4bits port map (i_S0 => w_S0,
                                      i_S1 => w_S1,
                                      o_Y0 => w_Y0,
                                      o_Y1 => w_Y1,
                                      o_Y2 => w_Y2,
                                      o_Y3 => w_Y3);

    process
    begin

```

```

-- Teste S1S0 = 00 -> Y0 = 1
w_S0 <= '0';
w_S1 <= '0';
wait for 1 ns;
assert (w_Y0 = '1' and w_Y1 = '0' and w_Y2 = '0' and w_Y3 = '0'
severity error;

-- Teste S1S0 = 01 -> Y1 = 1
w_S0 <= '1';
w_S1 <= '0';
wait for 1 ns;
assert (w_Y0 = '0' and w_Y1 = '1' and w_Y2 = '0' and w_Y3 = '0'
severity error;

-- Teste S1S0 = 10 -> Y2 = 1
w_S0 <= '0';
w_S1 <= '1';
wait for 1 ns;
assert (w_Y0 = '0' and w_Y1 = '0' and w_Y2 = '1' and w_Y3 = '0'
severity error;

-- Teste S1S0 = 11 -> Y3 = 1
w_S0 <= '1';
w_S1 <= '1';
wait for 1 ns;
assert (w_Y0 = '0' and w_Y1 = '0' and w_Y2 = '0' and w_Y3 = '1'
severity error;

-- Clear inputs
w_S0 <= '0';
w_S1 <= '0';
assert false report "Test done." severity note;
wait;
end process;

end tb;

```

7.4 Resultados da Simulação

A simulação foi realizada no ambiente EDA Playground. A Figura 4 apresenta as formas de onda obtidas durante a execução do testbench.

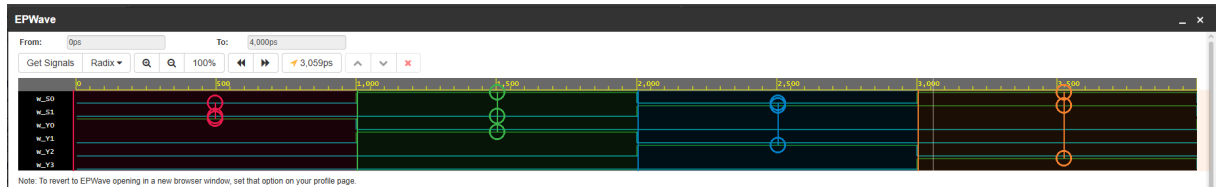


Figura 4: Formas de onda da simulação do conversor binário para one-hot.

Na seção em vermelho, podemos notar que: $S1S0=00$, temos $Y0=1$.

Na seção em verde, podemos notar que: $S1S0=01$, temos $Y1=1$.

Na seção em azul, podemos notar que: $S1S0=10$, temos $Y2=1$.

Na seção em laranja, podemos notar que: $S1S0=11$, temos $Y3=1$.

O resultado apresentado no console também foi analisado para verificar a execução dos testes.

7.5 Discussão dos Resultados

A simulação apresentou o comportamento esperado para o circuito. Para cada uma das quatro combinações binárias de $S1S0$, apenas a saída one-hot correspondente foi ativada, permanecendo as demais em nível lógico baixo. Os resultados observados nas formas de onda e no console confirmam o funcionamento correto do circuito.

8 CONCLUSÃO

A realização desta avaliação permitiu aplicar os conceitos de circuitos combinacionais por meio da descrição e simulação de diferentes circuitos utilizando VHDL. Foram analisados um multiplexador 2x1, um demultiplexador 1x2, um conversor de uma representação one-hot de 4 bits para uma representação binária de 2 bits e um conversor de uma representação binária de 2 bits para uma representação one-hot de 4 bits.

Os testbenches desenvolvidos permitiram verificar todas as combinações de entrada relevantes para cada circuito, utilizando asserções para comparar os valores obtidos com os resultados esperados. As formas de onda das simulações também permitiram observar o comportamento dos sinais de entrada e saída ao longo do tempo.

A partir dos resultados obtidos, foi possível verificar que os circuitos implementados apresentaram o comportamento lógico esperado, consolidando os conhecimentos relacionados à des-

crição de hardware em VHDL, elaboração de testbenches e análise de circuitos combinacionais por meio de simulação.